

What's new in C++26

News from the last C++ committee meeting

- Uninitialized reads are no longer UB. It's "erroneous behaviour."
- Policy discussions started.
 - `[[nodiscard]]` policy approved. Which is, don't `[[nodiscard]]` in the standard.
 - `noexcept` policy discussion started.
- `std::string + std::string_view`.
- `= delete("should have a reason");`
- ...and some other minor stuff.

Reminder: what's already in C++26

- `std::function_ref` and `std::copyable_function`.
- `pack...[indexing]`.
- `_placeholder`.
- Free function linear algebra interface based on BLAS.
- RCU & hazard pointers.
- `is_debugger_present()` & `breakpoint()`.
- ...and some other minor stuff.

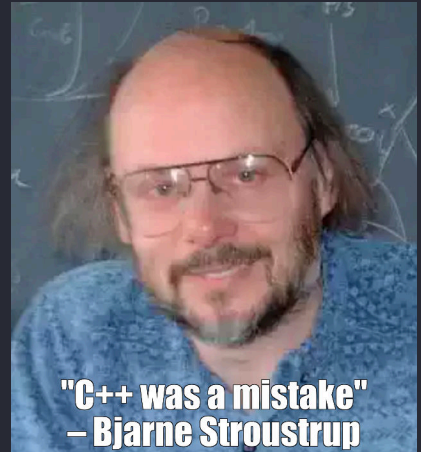
What C++26 is really about

Feature	Status	Current Target (Conservative)	Current Target (Optimistic)
Senders	Processed by LWG	C++26	C++26
Networking	Depends on Senders	C++29	C++26
Linear Algebra	Plenary approved	C++26	C++26
SIMD	Forwarded to LWG	C++26	C++26
Contracts	Processed in SG21	C++29	C++26
Reflection	Forwarded to EWG	C++26	C++26
Pattern Matching	Presented in EWG	C++29	C++26

What we mortal coders need

Greatest benefit to the community would be pattern matching. I did experiments 15 years ago, it has not been fashionable lately. We can't expect it to just appear, people need to work on that. I would like to see structured concurrency finished. We almost had it for 3 releases. After that maybe static reflection.

— Bjarne Stroustrup,
latest committee meeting.



This talk

1. Pattern Matching.
2. Reflection.
3. Lots of code.

I've read the proposals & sat through the discussions at the committee meeting, so that you don't have to.

Pattern matching

Pattern matching: the WHY

```
1  using Expression = std::variant<AddNode, MulNode, double>;
2
3  double evaluate(const Expression &e) {
4      return std::visit<double>([[<class T>(const T &arg) {
5          if constexpr (std::is_same_v<T, AddNode>)
6              return evaluate(*arg.l) + evaluate(*arg.r);
7          else if constexpr (std::is_same_v<T, MulNode>)
8              return evaluate(*arg.l) * evaluate(*arg.r);
9          else if constexpr (std::is_same_v<T, double>)
10             return arg;
11         else
12             static_assert(always_false_v<T>, "Non-exhaustive visitor!");
13     }, e);
14 }
```

C++ is a horrible language.

— Linus Torvalds, git mailing list.

Pattern matching: the WHY

```
1  using Expression = std::variant<AddNode, MulNode, double>;
2
3  double evaluate(const Expression &e) {
4      double value = 0.0;
5      if (std::holds_alternative<AddNode>(e))
6          value = evaluate(*std::get<AddNode>(e).l) + evaluate(*std::get<AddNode>(e).r);
7      else if (std::holds_alternative<MulNode>(e))
8          value = evaluate(*std::get<MulNode>(e).l) * evaluate(*std::get<MulNode>(e).r);
9      else if (std::holds_alternative<double>(e))
10         value = std::get<double>(e);
11     else
12         assert(false && "eeeh?");
13 }
```

Pattern matching: variants

```
1  struct Expression : std::variant<AddNode, MulNode, double> {};  
2  
3  double evaluate(const Expression &e) {  
4      return e match {  
5          AddNode: let arg ⇒ evaluate(*arg.l) + evaluate(*arg.r);  
6          MulNode: let arg ⇒ evaluate(*arg.l) * evaluate(*arg.r);  
7          double:  let arg ⇒ arg;  
8          -       ⇒ throw EvaluationException(); // e.valueless_by_exception().  
9      };  
10 }
```

- Matches are evaluated in order.
- Only expressions supported after $\Rightarrow$.
- Match expression `std::terminate()`s if nothing was matched.
- `let` to introduce a binding.

Pattern matching: std::any

```
1  using Expression = std::any;
2
3  double evaluate(const Expression &e) {
4      return e match {
5          AddNode: let arg => evaluate(arg.l) + evaluate(arg.r);
6          MulNode: let arg => evaluate(arg.l) * evaluate(arg.r);
7          double:  let arg => arg;
8          -       => throw EvaluationException(); // All other types.
9      };
10 }
```

Pattern matching: std::any

```
1  using Expression = std::any;
2
3  double evaluate(const Expression &e) {
4      return e match {
5          AddNode: let arg => evaluate(arg.l) + evaluate(arg.r);
6          MulNode: let arg => evaluate(arg.l) * evaluate(arg.r);
7          double:  let arg => arg;
8          -       => throw EvaluationException(); // All other types.
9      };
10 }
```

You don't have to worry about what is not important. That is the beauty of C++. It lets you concentrate on what you consider important.

— Bjarne Stroustrup, as hallucinated by ChatGPT.

Pattern matching: polymorphic classes

```
1  struct Expression { virtual ~Expression() = default; };
2
3  double evaluate(const Expression &e) {
4      return e match {
5          AddNode: let arg => evaluate(*arg.l) + evaluate(*arg.r);
6          MulNode: let arg => evaluate(*arg.l) * evaluate(*arg.r);
7          ValNode: let arg => arg.v;
8          -       => throw EvaluationException();
9      };
10 }
```

- There are techniques to make this more efficient than a sequence of `dynamic_casts`.

Pattern matching: switches

```
1  double evaluate(Operation op, double l, double r) {  
2      return op match {  
3          Operation::Add ⇒ l + r;  
4          Operation::Mul ⇒ l * r;  
5          Operation::Val ⇒ l;  
6          -              ⇒ throw EvaluationException();  
7      };  
8  }
```

- Labels are constant expressions.
- Matches are checked in order with operator`=`.
- Effectively compiles into a switch.

Pattern matching: better than a switch

```
1  Color colorFromString(std::string_view s) {  
2      return s match {  
3          "red"    ⇒ Color::Red;  
4          "green"  ⇒ Color::Green;  
5          "blue"   ⇒ Color::Blue;  
6          "orange" ⇒ Color::Orange;  
7          _       ⇒ Color::Unknown;  
8      };  
9  }
```

- Remember that this is evaluated in order.
- There are opportunities for optimization if we allow compiler magic here.

Pattern matching: tuple-like objects

```
1  double evaluate(const std::tuple<Operation, double, double> &op) {  
2      return op match {  
3          [Operation::Add, let l, let r] => l + r;  
4          [Operation::Mul, let l, let r] => l * r;  
5          [Operation::Val, let v, let _] => v;  
6          -                               => throw EvaluationException();  
7      };  
8  }
```

- Code is retarded, for illustrative purposes only.
- Shows why we need `let`.

Pattern matching: structured bindings

```
1  double classify(Point point) {  
2      return point match {  
3          [0, 0]      ⇒ PointClass::Origin;  
4          [0, let _]  ⇒ PointClass::YAxis;  
5          [let _, 0]  ⇒ PointClass::XAxis;  
6          -           ⇒ PointClass::Other;  
7      };  
8  }
```

- This is C++ and we can't have nice things, so `_`s in the code are conceptually different.

Pattern matching is composable!

```
1  using Expression = std::any;
2
3  double evaluate(const std::any &e) {
4      return e match {
5          AddNode: let [l, r] => evaluate(l) + evaluate(r);
6          MulNode: let [l, r] => evaluate(l) * evaluate(r);
7          double:  let v      => v;
8          -        => throw EvaluationException();
9      };
10 }
```

- Also possible:
 - let [[a, b], c].
 - [[0, 0], let c].
 - [MyPair: let [a, b], 0].

Pattern matching: ifs everywhere!

```
1  int fib(int n) {  
2      return n match {  
3          let x if (x < 0) => 0;  
4          1 => n;  
5          2 => n;  
6          let x => fib(x - 1) + fib(x - 2);  
7      };  
8  }
```

```
1  if (expr match [let foo, 0]) {  
2      // `foo` is available here  
3  }
```

Pattern matching: extension points

- `operator=` for constant expression matches.
- `std::tuple_size<T>`, `std::tuple_element<I, T>`, ADL `get<I>(v)` for tuple-like types.
- `std::variant_size<T>`, `std::variant_alternative<I, T>`, ADL `index(v)` and `get<I>(v)` for variant-like types.
- ADL `cast<T>(v)` for `std::any`, `std::exception_ptr`, polymorphic types.

Pattern matching: open questions

- Matching `std::variant<T, T>`.
- Matching `std::expected<T, T>`.
- `std::terminate` if we match nothing is a bit... excessive?
 - Do we do anything with non-exhaustive match expressions at compile time?
- Fully specified extension protocols. Right now variant protocol isn't really a thing (`std::visit` isn't even implemented in terms of this "protocol").

Reflection

Reflection: prior art

- Started `pre-constexpr`, when we only had template-based metaprogramming.
- Reflection TS with `reflexpr` goes way back to 2010s.
- People have been having fun for a long time:
 - `Boost.Describe`.
 - `Boost.PFR`.
 - `refl-cpp`.
 - ...
- Design settled on `constexpr` functions, and C++ didn't have these for a long time (`constexpr` is C++20, same as `constexpr std::vector`).

Reflection: reflecting & unreflecting

```
1  constexpr std::meta::info r = ^int;  
2  [:r:] value = 42;  
3  
4  std::vector<[:r:]> v = { value };  
5  
6  [:^(vector<int>):]::iterator it;
```

- `std::meta::info` is an opaque type for reflection.
- Prefix `^` reflects.
- Splacers in the form `[ : refl : ]` produce grammatical elements from reflections.
"Accordion operator."

Reflection: reflecting & unreflecting

```
1  constexpr std::meta::info v = ^std::vector;  
2  constexpr std::meta::info a = ^int;  
3  constexpr std::meta::info va = std::meta::substitute(v, {a});  
4  
5  [:va:] vec = {1, 2, 3}; // Got std::vector<int> back.
```

- Can reflect & unreflect templates.
- A number of `constexpr` functions are offered that work with `std::meta::info`.

Reflection: member access

```
1  struct Person {
2      std::string name;
3      std::string surname;
4  };
5
6  constexpr auto member_named(std::string_view name) {
7      for (std::meta::info field : nonstatic_data_members_of(^Person)) {
8          if (name_of(field) == name) return field;
9      }
10 }
11
12 int main() {
13     Person p;
14     p[:member_named("name"):] = "John";
15     p[:member_named("surname"):] = "Doe";
16     p[:member_named("nickname"):] = "Coder"; // Error.
17 }
```

Reflection: codegen with template for

```
1  using StringHandler = std::string(*)(const void *);
2
3  template<class T>
4  static std::string stringize(const void *ptr) {
5      return std::to_string(*static_cast<const T *>(ptr));
6  }
7
8  constexpr std::array types = {^int, ^float, ^double}; // And maybe more.
9
10 constexpr std::array handlers = [] {
11     std::array<StringHandler, types.size()> result;
12     template for (std::size_t i = 0; constexpr auto e : types)
13         result[i++] = &stringize<typename[e:]>;
14     return result;
15 }();
```

- Today you would normally do this with macros or type lists.

Reflection: enum to string

```
1  template <typename E>
2      requires std::is_enum_v<E>
3  constexpr std::string enum_to_string(E value) {
4      template for (constexpr auto e : std::meta::enumerators_of(^E)) {
5          if (value == [:e:]) {
6              return std::string(std::meta::name_of(e));
7          }
8      }
9
10     return "<unnamed>";
11 }
12
13 enum Color { red, green, blue };
14 static_assert(enum_to_string(Color::red) == "red");
15 static_assert(enum_to_string(Color(42)) == "<unnamed>");
```

Reflection: string to enum

```
1  template <typename E>
2      requires std::is_enum_v<E>
3  constexpr std::optional<E> string_to_enum(std::string_view name) {
4      template for (constexpr auto e : std::meta::enumerators_of(^E)) {
5          if (name == std::meta::name_of(e)) {
6              return [:e:];
7          }
8      }
9
10     return std::nullopt;
11 }
```

Reflection: string to enum, but better

```
1  template<class E>
2      requires std::is_enum_v<E>
3  constexpr make_enum_pairs() {
4      constexpr auto size = std::meta::enumerators_of(^E).size();
5      std::array<std::pair<std::string_view, E>, size> result;
6      for (std::size_t i = 0; auto e : std::meta::enumerators_of(^E)) {
7          result[i++] = {name_of(e), value_of<E>(e)};
8      }
9      return result;
10 }
11
12 enum Color { red, green, blue };
13 constexpr auto mapping = frozen::make_unordered_map(make_enum_pairs<Color>());
```

Reflection: tuple

```
1  template<typename... Ts> struct Tuple {
2      struct storage;
3      [:define_class(^storage, { data_member_spec(^Ts)... }):] data;
4      Tuple(): data{} {}
5      Tuple(Ts const& ...vs): data{ vs... } {}
6  };
7
8  template<std::size_t I, typename... Ts>
9  struct std::tuple_element<I, Tuple<Ts...>> {
10     static constexpr std::array types = {^Ts...};
11     using type = [: types[I] :]; // Or you can do pack...[indexing].
12 };
13
14 constexpr std::meta::info get_nth_nsdm(std::meta::info r, std::size_t n) {
15     return nonstatic_data_members_of(r)[n];
16 }
17
18 template<std::size_t I, typename... Ts>
19 constexpr auto get(Tuple<Ts...> &t) noexcept → std::tuple_element_t<I, Tuple<Ts...>& {
20     return t.data.[:get_nth_nsdm(^decltype(t.data), I):];
21 }
```

Reflection: hash_append

```
1  template <typename H, typename T>
2      requires std::is_standard_layout_v<T>
3  void hash_append(H& algo, T const& t) {
4      template for (constexpr auto mem : nonstatic_data_members_of(^T)) {
5          hash_append(algo, t.[:mem:]);
6      }
7  }
```

Reflection: fun

- Reflection observes compilation state.
- Reflection modifies compilation state.

```
1  constexpr auto type = ^std::vector<int>;
2
3  // This requires instantiation!
4  constexpr auto members = nonstatic_data_members_of(type);
5
6  // And we can check whether the type was instantiated...
7  constexpr bool fun = is_incomplete_type(type);
```

Thus, you can make a counter (like the `__COUNTER__` macro) on reflection. Could do it before with declarations, now it's just a `constexpr` function call.

Reflection: open questions

What does this code do?

```
1 constexpr auto tmp1 = template_of(^int);
```

- Throws?
- Returns `std::expected`?
- Returns an empty `std::meta::info`?
- Some other mechanism? Fails to compile with "not-a-constant-expression"?

Would be nice to do this with exceptions, need `constexpr` exceptions in the language.

Reflection: open questions

- Current proposal doesn't solve the codegen question.
 - Template `switch`, anyone?
- `define_class` API in its current form is questionable.
 - How do we add member functions?
 - How do we add `[[no_unique_address]]` to a member?

END